CO2- Assessment-02

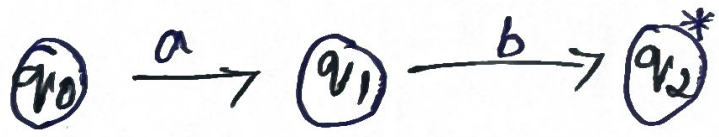
E. Gopi Krishna
- 19231347

① Regular expression from Automata

1. Identify start & final states
2. Trace all possible paths
3. combine paths using:
   - Union (+)
   - Concatenation
   - Kleene Star (*)

State

Ex:-



Regex = ab

② Grammar

$S \rightarrow aSc \mid T \mid U \mid \varepsilon$

$T \rightarrow bTc \mid \varepsilon$

$U \rightarrow aUb \mid \varepsilon$

Language includes:

- equal no. of a and c
- equal no. of b and c

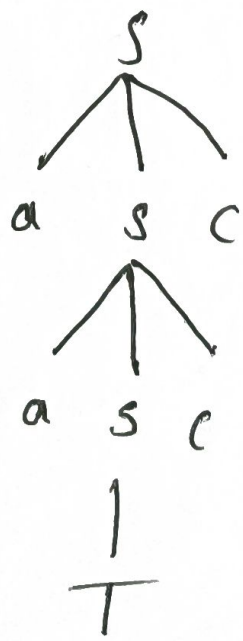
- equal no. of a and b

So strings like:

$a^n c^n$, $b^n c^n$, $a^n b^n$, ε

(B) Derivation Tree for " aaabbcc"

$S \rightarrow aSc$
$\rightarrow aaScc$
$\rightarrow aaaSccc$
$\rightarrow aaaTccc$
$\rightarrow aaabTcccc$
$\rightarrow aaabbtccccc$
$\rightarrow aaabbc$



© Ambiguity

Grammar is Ambiguous

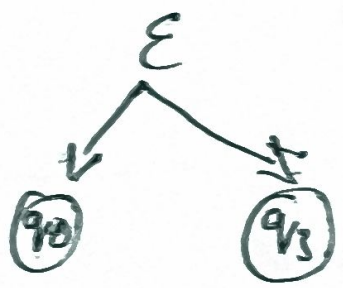
Rem :-

- Same string can be derived using:
  - $S \rightarrow T$
  - $S \rightarrow U$

③ ε – NFA for Regex

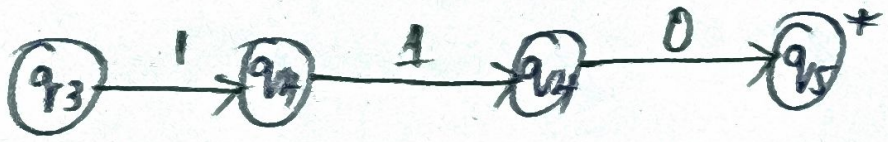
Given

$00^*1 + 11^*0$



Path 1: 00^*1

Path 2: 11^*0



Final State:

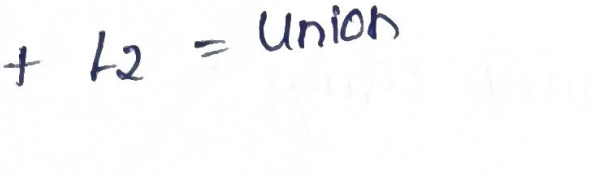
q_2 , q_5

$\delta(\{q_0\}, b) = \{q_2\}$

Q4) Closure property

If L_1 and L_2 are regular

$L_1 + L_2$ = Union



because DFA/NFA can combine both

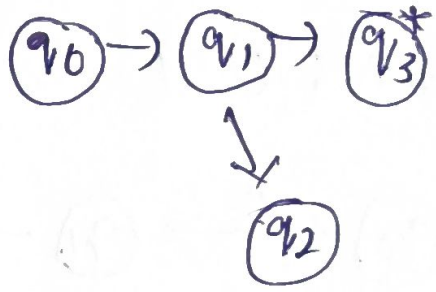
(ii) Language { a^p | p is prime }

- prime numbers cannot be mainted by finite memory
- FA cannot count primes

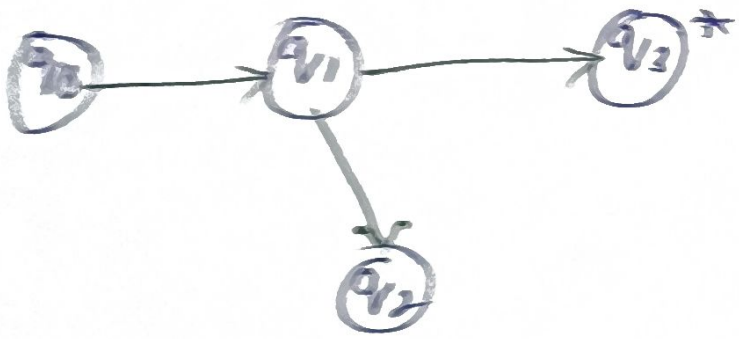
So language is not regular

(5) DFA Minimalisation

1. Seperate final & non final
2. Check equivalent states
3. Merge equivalent states
4. Draw minimized DFA



State	0	1
q_0	q_1	q_2
q_1	q_1	q_3
q_2	q_1	q_2
q_3	q_3	q_3



Reduced no. of states → efficient

Final Conclusion

Regular expressions ⟷ Automata

Grammar helps define languages

ε-NFA makes design easier

Some languages (prime, $a^n b^m$) are not regular

Minimization improves performance.